

p1229R0

Labelled Parameters

Published Proposal, 2018-10-08

This version:

<http://wg21.link/p1229r0>

Author:

[Jorg Brown](#) (Google)

Audience:

EWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

This paper proposes a method for giving names to function parameters, compatible with `std::forward`'ing and without impact on existing code.

Table of Contents

- 1 Change history**
- 2 Motivating examples and previous attempts**
 - 2.1 Ordering of similarly-typed parameters
 - 2.2 Constructor overloads
 - 2.3 Previous proposals
- 3 Approach**
 - 3.1 Parameter names map 1:1 to empty classes
 - 3.2 Labelled parameters derive from those empty classes
 - 3.3 One more thing - explicit labelled parameters

4 Syntactic Sugar

4.1 Labels in a Function Declaration

4.1.1 The function call

4.1.2 Within the context of a function declaration, the parameter-declaration:

4.1.3 Within the context of a function declaration, the explicit parameter-declaration:

4.1.4 Within the context of a function definition, the parameter name:

5 How close can we get today?

6 Acknowledgements

References

Informative References

§ 1. Change history

This is the initial version, [\[P1229R0\]](#)

§ 2. Motivating examples and previous attempts

§ 2.1. Ordering of similarly-typed parameters

Consider the following code:

```
void Process(char *in, size_t bytes) {  
    char *buf = new char[bytes + 1];  
    std::memcpy(in, buf, bytes);  
    buf[bytes] = '\0';  
    ProcessCString(buf);  
}
```

Due to its author's previous use of `std::copy_n`, where the source is the first parameter, the author here has also placed the input as the first parameter, not realizing that `memcpy`'s arguments aren't in that order. Furthermore since the author has neglected to `const`-qualify the routine's "in" parameter, the mistake is not caught at compile time.

In other languages, such as python and Kotlin, parameters can be named, which allows for compile-time detection of errors. However their syntax conflicts with existing meaning since `=` can be used in any C++ expression:

```
std::memcpy(dst=buf, src=in, n=bytes); // three assignments and a function call
```

Clang-tidy, among others, allows for inline documentation of parameter names, and will generate warnings if the documentation does not match the parameter names used in the routine's declaration.

```
std::memcpy(/* src= */ buf, /* dst= */ in, /* n= */ bytes); // Clang-tidy warning
std::memcpy(/* s1= */ buf, /* s2= */ in, /* n= */ bytes); // No problem
```

However the standard does not specify what the header file must use for parameter names, so this results in code that may not be compatible with all library implementations. Choices made in existing APIs are sometimes unfortunate; C++17 has chosen "s1", "s2", and "n" as the parameter names for `memcpy`, which is why clang-tidy sees no problem with the second line above.

Also, such parameter comments do not survive a `std::forward` call, so while there may be safety in using these name comments in most function calls, the name comments would be ignored in a call to `std::make_shared()`.

§ 2.2. Constructor overloads

The need for named parameters is most acute in situations where functions are most overloaded: constructors. For `std::pair`, in fact, this was solved by adding an empty struct tag type, `std::piecewise_construct_t`, to select a particular overload. Similarly, `std::allocator_arg_t` is used for disambiguation in the constructors for `std::tuple`, `std::function` and others.

§ 2.3. Previous proposals

[\[N4172\]](#) "Named Arguments" also proposed a mechanism for naming arguments, with a function-call syntax identical to that proposed in this paper. However:

1. The declaration syntax was identical to current C++, basing the parameter names callers must use on the names chosen in header files. This has the unfortunate property of making currently-arbitrary naming choices part of the API for future callers. This paper adopts the view that

labelling parameters should be an "opt-in" choice, and should not by default apply to existing declarations. N4172 claims that breakage due to parameter name changes would be low, because parameter names don't change that often. A bigger problem is that the very same API is often declared with different names, by different library vendors. For example, `memcpy`'s parameters are most often declared as `dest / src / n`, but `dst / src / n` is sometimes used, as is `__dst / __src / __n`, and as noted above the C++ standard itself (and POSIX) uses `s1 / s2 / n`.

2. N4172 changes the meaning and validity of existing code when the name given to parameters in the function declaration, and the function definition, don't match. However such mismatches are quite common and may occur in library code which a user has no control over.
3. N4172 does not allow labelled parameters to be part of a variadic template function call.
4. N4172 does not allow labelled parameters to be part of function-pointer syntax.

§ 3. Approach

For labelled parameters, this proposal adopts the empty-struct-tag approach, with one minor change: rather than a separate parameter with the tag type, we subclass the empty tag type, and add the actual parameter as a member of the subclass.

§ 3.1. Parameter names map 1:1 to empty classes

The language feature needed most by this proposal is a way to define a new type on the fly when a function is declared with a labelled parameter. Ideally there would be a way to pass a literal string as a template parameter; then we could write `std::label<"from">` to denote that type. Instead, we accomplish that with individual template arguments:

```
template <typename CharT, CharT... String> class label;
```

Now we can write `std::label<char, 'f', 'r', 'o', 'm'>` to get a unique type name for purposes of forming a unique argument type.

§ 3.2. Labelled parameters derive from those empty classes

To support optional use, a labelled parameter should be implicitly constructible from its unlabelled type; it must also be constructible from a labelled parameter whose type does not exactly match. So it looks like this:

```

template <typename Label, typename T>
struct labelled : public Label {
    T value;

    template <typename U, typename = typename std::enable_if<
        std::is_convertible<U&&, T>::value>::type>
    constexpr labelled(U&& u) : value(std::forward<U>(u)) {}

    template <typename U>
    constexpr labelled(labelled<Label, U>&& ref)
        : value(std::forward<decltype(ref.value)>(ref.value)) {}
};

```

This is already enough to start using labelled parameters, however the syntax is unwieldy:

```

// declaration
void memcpy(std::labelled<std::label<char, 't', 'o'>, char *>,
            std::labelled<std::label<char, 'f', 'r', 'o', 'm'>, const char
            std::labelled<std::label<char, 'n'>, size_t>);
// call site
memcpy(std::labelled<std::label<char, 't', 'o'>, char *>(buf),
        std::labelled<std::label<char, 'f', 'r', 'o', 'm'>, const char *>(ir
        std::labelled<std::label<char, 'n'>, size_t>(bytes));

```

We can improve this situation by adding a custom type and an operator() to our formerly-empty label class:

```

template <typename CharT, CharT... String>
class label {
public:
    template <typename T>
    using param = labelled<label, T>;

    template <typename T>
    constexpr labelled<label, T> operator()(T&& t) const {
        return {std::forward<T>(t)};
    }
};

```

And now the code is a bit more readable:

```
// declaration
void memcpy(std::label<char, 't', 'o'>::param<char *>,
            std::label<char, 'f', 'r', 'o', 'm'>::param<const char *>,
            std::label<char, 'n'>::param<size_t>);
// call site
memcpy(std::label<char, 't', 'o'>()(buf),
        std::label<char, 'f', 'r', 'o', 'm'>()(in),
        std::label<char, 'n'>()(bytes));
```

§ 3.3. One more thing - explicit labelled parameters

In some cases, the automatic conversion of a non-labelled parameter to a labelled function argument type may be undesirable. For example, in adding an overload to an existing function that accepts a pointer, if the new overload accepts an integer, then any call with the parameter 0 as an argument becomes ambiguous. But if the new overload uses an explicitly labelled parameter, then there will be no ambiguity. The new class is even simpler:

```
template <typename Label, typename T>
struct explicit_labelled : public Label {
    T value;

    template <typename U>
    constexpr explicit_labelled(labelled<Label, U>&& ref)
        : value(std::forward<decltype(ref.value)>(ref.value)) {}
};
```

§ 4. Syntactic Sugar

While functional, the technique we have described thus far is ugly enough that most programmers would not use it. With language support it becomes much cleaner.

§ 4.1. Labels in a Function Declaration

§ 4.1.1. The function call

```
function-name( labelname : expression )
```

is syntactically equivalent to

```
function-name( std::label<char, 'l', 'a', 'b', 'e', 'l', 'n', 'a', 'm',  
'e'>()(expression) )
```

§ 4.1.2. Within the context of a function declaration, the parameter-declaration:

```
labelname : attribute-specifier-seq-opt decl-specifier-seq declarator =  
initializer-clause
```

is syntactically equivalent to

```
attribute-specifier-seq-opt std::labelled<std::label<char, 'l', 'a', 'b',  
'e', 'l', 'n', 'a', 'm', 'e'>, decl-specifier-seq> declarator = initializer-  
clause
```

§ 4.1.3. Within the context of a function declaration, the explicit parameter-declaration:

```
explicit labelname : attribute-specifier-seq-opt decl-specifier-seq  
declarator = initializer-clause
```

is syntactically equivalent to

```
attribute-specifier-seq-opt std::explicit_labelled<std::label<char, 'l',  
'a', 'b', 'e', 'l', 'n', 'a', 'm', 'e'>, decl-specifier-seq> declarator =  
initializer-clause
```

§ 4.1.4. Within the context of a function definition, the parameter name:

actually refers to the "value" field of the std::labelled struct.

This works as though the parameter had actually had a different name, and then within the function, the parameter was defined to refer to the value field. So the definition line:

```
function-name(labelname : attribute-specifier-seq-opt decl-specifier-seq
declarator) {
```

becomes:

```
function-name(labelname : attribute-specifier-seq-opt decl-specifier-seq __
    auto &&declarator = __declarator.value;
```

where `__declarator` is an implementation-defined renaming of "declarator".

With this sugar, the `memcpy`-with-labels declaration and call becomes quite clean, even familiar to users of languages such as C# with similar syntax:

```
// declaration
void memcpy(to: char *, from: const char *, n: size_t);
// call site
memcpy(to: buf, from: in, n: bytes);
```

§ 5. How close can we get today?

We can use the `std::label(char, 'l', 'a', 'b', 'e', 'l')` syntax today, to answer various questions about how this proposal works... but that gets a bit unwieldy. Fortunately we can do a little better: [\[N3599\]](#), "Literal operator templates for strings" provides a convenient mechanism currently implemented in both gcc and clang (though not officially part of C++).

```
template <typename CharT, CharT... String>
constexpr std::label<CharT, String...> operator""_label() {
    return {};
}
```

Now we can write `decltype("from"_label)` to get the unique type name we need; the `memcpy` example becomes much more reasonable:


```

void memcpy(decltype("to"_label)::param<char *>,
            decltype("from"_label)::param<const char *>,
            decltype("n"_label)::param<size_t>);

// ...

void Process(char *in, size_t bytes) {
    char *buf = new char[bytes + 1];
    memcpy(decltype("to"_label)(buf), decltype("from"_label)(in), decltype("n"_label)(bytes));
    buf[bytes] = '\0';
    ProcessCString(buf);
}

```

One can see these in practice at [godbolt QfIkKhO](http://godbolt.com/QfIkKhO), which also serves to answer many questions regarding parameter matching, overload resolution, use within lambdas, `std::forward` propagation, etc.

§ 6. Acknowledgements

Thanks to Richard Smith for pointing me at N3599, and providing various other insights. Thanks to Matthew Godbolt whose Compiler Explorer sped up the development of this proposal significantly.

§ References

§ Informative References

[N3599]

Richard Smith. [Literal operator templates for strings](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2013/n3599.html). URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2013/n3599.html>

[N4172]

Ehsan Akhgari; Botond Ballo. [Named arguments](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4172.htm). URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4172.htm>

[P1229R0]

Jorg Brown. [Labelled Parameters](http://wg21.link/p1229r0). URL: <http://wg21.link/p1229r0>